



Rapport projet de BDD

Groupe 7

Rayane HAMMIDA
Sohaib EL KAICH
Matéo DELFORGE
Simon DE CUBBER

Faculté d'informatique
INFOB212
Année académique : 2025/2026

Introduction.....	3
Description du domaine d’application:.....	4
Schéma conceptuel.....	5
Relations et cardinalités.....	6
Schéma logique et schéma physique.....	8
Types numériques.....	9
Types textuels.....	9
Type date.....	9
Implémentation de l'Application.....	10
1. Choix Technologiques.....	10
2. Architecture Logicielle.....	10
3. Fonctionnalités Couvertes.....	11
Implémentation SQL :.....	11
Conclusion.....	15

Introduction

Dans le cadre de la deuxième partie du cours de base de données, ce rapport présente la conception, le déploiement et l'exploitation d'une base de données relationnelle dédiée à la gestion d'une agence de location de voitures.

Notre objectif à travers ce projet est de concevoir un système complet et robuste permettant de gérer le cycle de vie complet d'une location : de l'inscription du client à la restitution du véhicule, en passant par la gestion comptable.

Ce document détaillera dans un premier temps notre modélisation de la base de données (du schéma conceptuel au schéma physique), puis explicitera nos choix technologiques pour le déploiement. Enfin, nous présenterons l'application logicielle développée, illustrant comment celle-ci interagit avec la base de données tout en respectant des contraintes strictes telles que la sécurité des accès et le règlement général sur la protection des données (RGPD).

Description du domaine d'application:

Le domaine choisi pour ce projet est celui d'une entreprise de location de voitures. Le système permet de gérer les agences, les véhicules, les clients, les réservations, les locations, les paiements, les frais supplémentaires ainsi que les employés de l'entreprise.

L'entreprise est organisée autour de plusieurs agences. Chaque agence possède un nom, une adresse, une ville, un code postal et un numéro de téléphone. Une agence peut employer plusieurs employés. Chaque employé est identifié par son nom, son prénom, son email, son téléphone optionnel et sa date d'embauche. Les employés sont séparés en deux rôles : les agents d'agence et les comptables. Les agents d'agence s'occupent principalement de la gestion des clients, des réservations et des locations, tandis que les comptables s'occupent des paiements et des rapports financiers.

Les véhicules proposés à la location sont décrits par leur immatriculation, leur marque, leur modèle, leur année, leur kilométrage actuel, leur état et leur tarif journalier. Chaque véhicule appartient à une seule catégorie de véhicule, par exemple citadine, SUV ou utilitaire. Une catégorie possède un nom, une description optionnelle et un tarif de base. Chaque véhicule est également rattaché à une agence, ce qui permet de savoir dans quelle agence il se trouve.

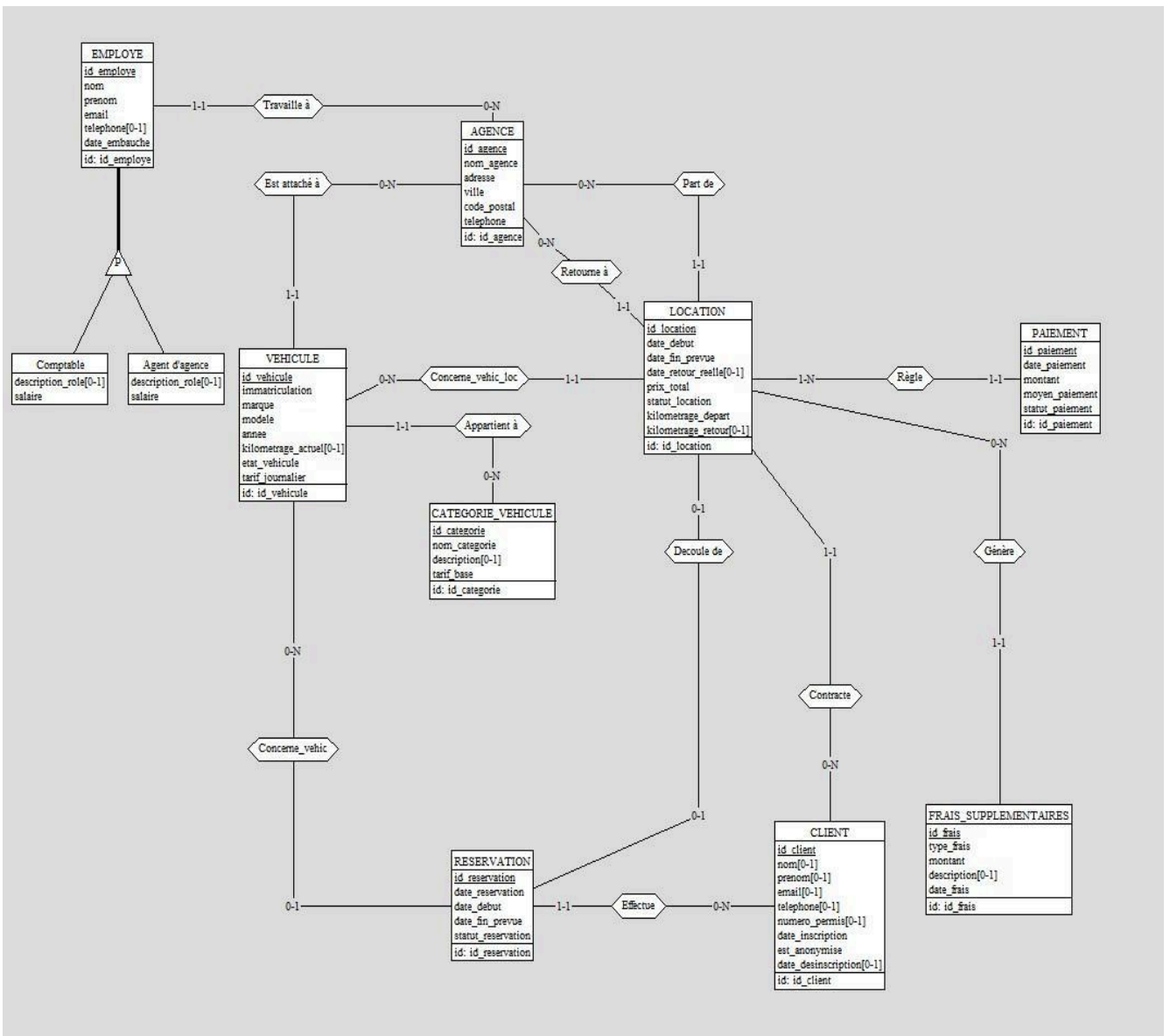
Les clients peuvent s'inscrire dans le système avec leurs informations personnelles : nom, prénom, email, téléphone, numéro de permis et date d'inscription. Une fois inscrits, ils peuvent effectuer une ou plusieurs réservations. Dans notre version finale, une réservation concerne toujours un véhicule précis et une période donnée. Elle contient une date de réservation, une date de début, une date de fin prévue et un statut de suivi. Ce choix permet d'éviter une réservation trop générale par catégorie et facilite la vérification de la disponibilité réelle d'un véhicule.

Une réservation peut ensuite donner lieu à une location effective. Une location est associée à un client, à un véhicule, à une agence de départ et à une agence de retour. Elle contient les dates de location, le prix total, le statut de la location, le kilométrage de départ et éventuellement le kilométrage de retour. Une location peut découler d'une réservation, mais elle peut aussi être créée directement si un client se présente en agence sans réservation préalable.

Chaque location peut être réglée par un ou plusieurs paiements. Un paiement contient une date, un montant, un moyen de paiement et un statut. Cette séparation permet de gérer le cas où une location est payée en plusieurs fois. Une location peut aussi générer des frais supplémentaires, par exemple en cas de retard, de dommages ou de dépassement de kilométrage. Chaque frais supplémentaire possède un type, un montant, une description optionnelle et une date.

Enfin, le système prend en compte la désinscription des clients. Lorsqu'un client se désinscrit, ses données personnelles sont anonymisées afin qu'il ne soit plus possible de retrouver son identité à partir de la base. En revanche, l'historique de ses locations, paiements et frais reste conservé pour les besoins de suivi de l'entreprise. Ce choix permet de respecter la protection des données personnelles tout en gardant les informations utiles à l'activité de location.

Schéma conceptuel



Relations et cardinalités

1)-EMPLOYE → AGENCE — Travaille à

Cardinalité : 1-1 → 0-N

Un employé travaille dans une seule agence. Une agence peut employer zéro ou plusieurs employés.

2)-EMPLOYE → COMPTABLE / AGENT_D_AGENCE — Héritage

Type : spécialisation exclusive

Un employé peut être spécialisé soit comme comptable, soit comme agent d'agence. Ces deux rôles sont séparés afin de distinguer les employés chargés de la gestion des locations et ceux chargés de la partie financière.

3)-VEHICULE → AGENCE — Est attaché à

Cardinalité : 1-1 → 0-N

Un véhicule est rattaché à une seule agence. Une agence peut gérer zéro ou plusieurs véhicules.

3)-VEHICULE → CATEGORIE_VEHICULE — Appartient à

Cardinalité : 1-1 → 0-N

Un véhicule appartient à une seule catégorie. Une catégorie peut regrouper zéro ou plusieurs véhicules.

4)-LOCATION → AGENCE — Part de

Cardinalité : 1-1 → 0-N

Une location démarre depuis une seule agence. Une agence peut être l'agence de départ de zéro ou plusieurs locations.

5)-LOCATION → AGENCE — Retourne à

Cardinalité : 1-1 → 0-N

Une location se termine dans une seule agence de retour. Une agence peut accueillir zéro ou plusieurs retours de véhicules.

6)-LOCATION → VEHICULE — Concerne_vhic_loc

Cardinalité : 1-1 → 0-N

Une location concerne un seul véhicule. Un véhicule peut être concerné par zéro ou plusieurs locations, mais pas sur des périodes qui se chevauchent.

7)-LOCATION → PAIEMENT — Règle

Cardinalité : 1-N → 1-1

Une location peut être réglée par un ou plusieurs paiements. Chaque paiement correspond à une seule location. Ce choix permet de gérer le cas où une location est payée en plusieurs fois.

8)-LOCATION → FRAIS_SUPPLEMENTAIRES — Génère

Cardinalité : 1-1 → 0-N

Une location peut générer zéro ou plusieurs frais supplémentaires. Chaque frais supplémentaire est lié à une seule location.

9)-LOCATION → RESERVATION — Découle de

Cardinalité : 0-1 → 0-1

Une location peut découler d'une réservation, mais elle peut aussi être créée directement sans réservation préalable. Une réservation peut ne pas encore avoir donné lieu à une location.

10)-LOCATION → CLIENT — Contracte

Cardinalité : 1-1 → 0-N

Une location est contractée par un seul client. Un client peut contracter zéro ou plusieurs locations.

11)-RESERVATION → CLIENT — Effectue

Cardinalité : 1-1 → 0-N

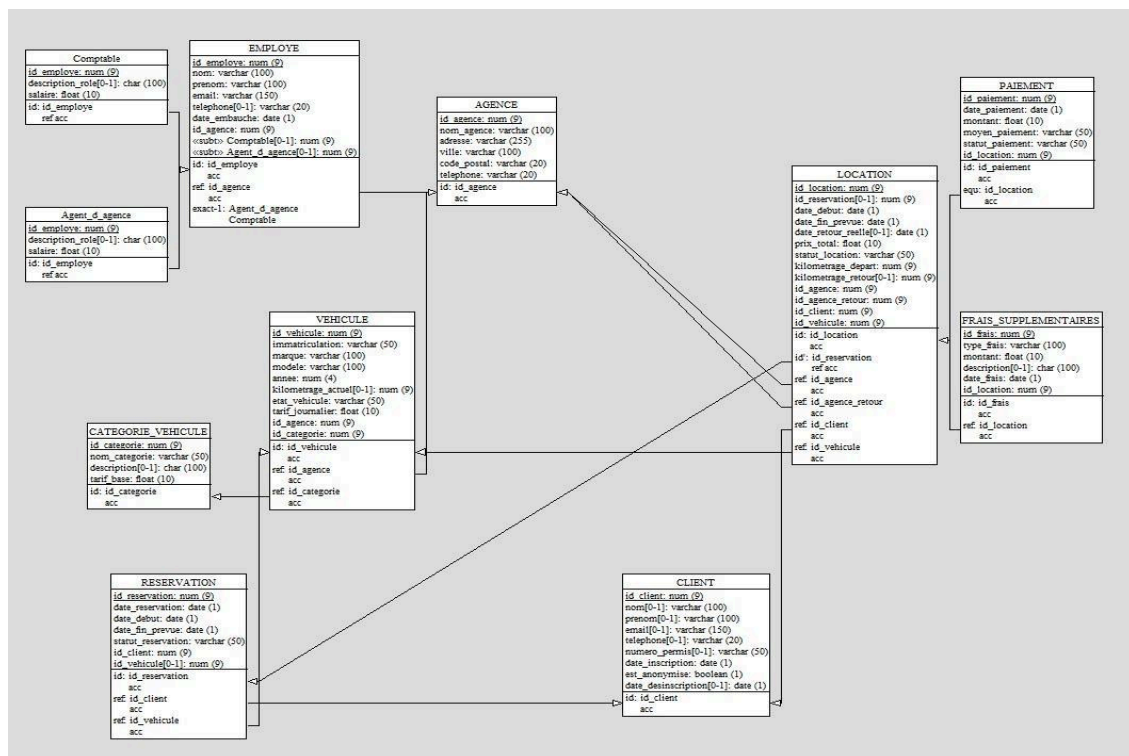
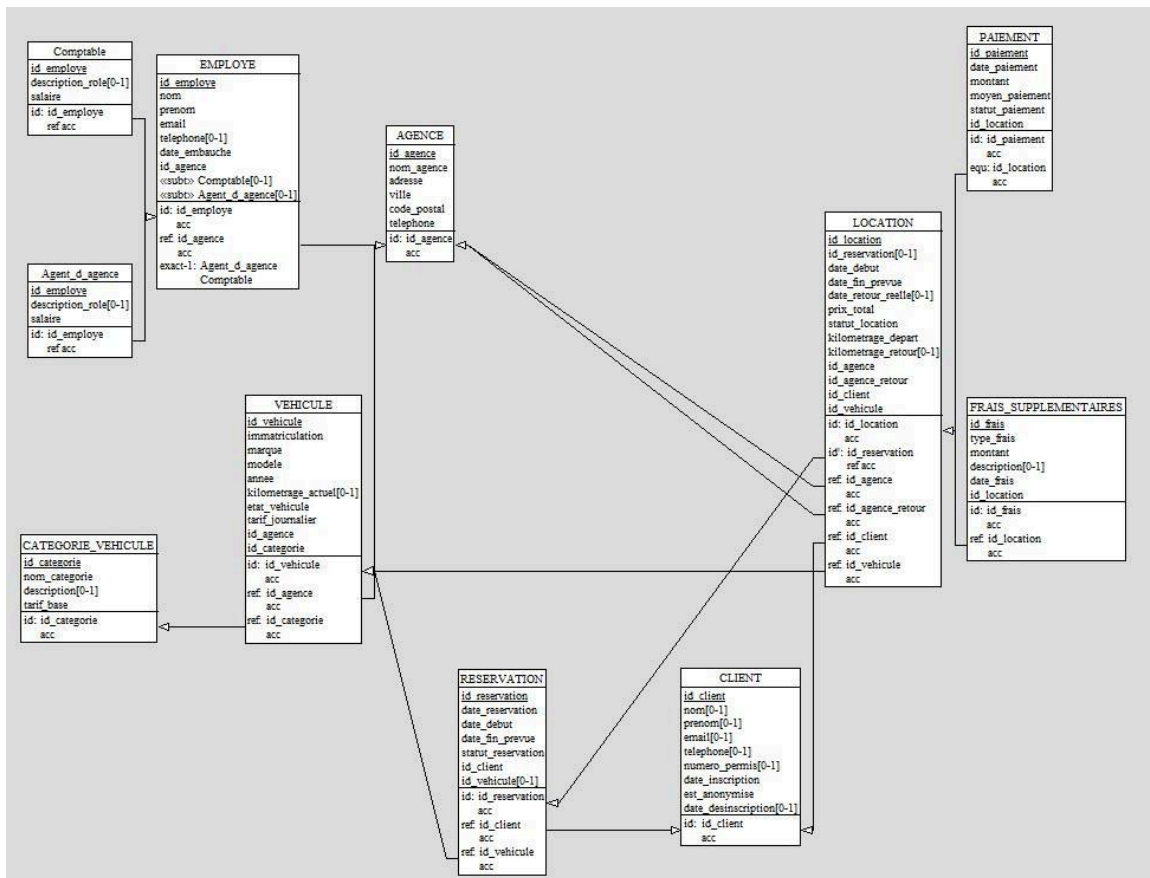
Une réservation est effectuée par un seul client. Un client peut effectuer zéro ou plusieurs réservations.

12)-RESERVATION → VEHICULE — Concerne_vehic

Cardinalité : 1-1 → 0-N

Une réservation concerne exactement un véhicule précis. Un véhicule peut être visé par zéro ou plusieurs réservations, à condition que les périodes ne se chevauchent pas.

Schéma logique et schéma physique



Types numériques

-INT est utilisé pour tous les identifiants primaires et étrangers, comme id_client, id_vehicule, id_location, id_agence, etc. Ce type est suffisant pour identifier les enregistrements de chaque table.

-INT est aussi utilisé pour les valeurs entières comme annee, kilometrage_actuel, kilometrage_depart et kilometrage_retour. Ces informations ne nécessitent pas de décimales.

-DECIMAL(10,2) est utilisé pour les montants financiers, comme prix_total, montant, tarif_journalier, tarif_base et salaire. Ce type permet de stocker des valeurs avec deux chiffres après la virgule, ce qui est adapté pour les prix et les paiements.

-BOOLEAN est utilisé pour est_anonymise dans la table Client. Il permet de savoir si les données personnelles d'un client ont été anonymisées ou non.

Types textuels

-VARCHAR est utilisé pour les textes courts ou de longueur variable, comme les noms, prénoms, emails, téléphones, marques, modèles, statuts, moyens de paiement et immatriculations.

-VARCHAR(100) est utilisé pour les champs comme nom, prenom, marque, modele, nom_agence et nom_categorie, car ces valeurs restent généralement courtes.

-VARCHAR(150) est utilisé pour email, car une adresse email peut être plus longue qu'un simple nom.

-VARCHAR(255) est utilisé pour adresse, car une adresse complète peut être plus longue et contenir plusieurs informations.

-VARCHAR(50) est utilisé pour les champs comme statut_location, statut_paiement, etat_vehicule, moyen_paiement, immatriculation et numero_permis. Ces valeurs sont variables mais restent limitées.

-VARCHAR(20) est utilisé pour telephone et code_postal, car ces informations peuvent contenir des zéros au début, des espaces ou des caractères spéciaux. Elles ne doivent donc pas être stockées comme des nombres.

-TEXT est utilisé pour les descriptions, comme description, description_role ou la description d'un frais supplémentaire. Ce choix permet de stocker des textes plus longs si nécessaire.

Type date

-DATE est utilisé pour toutes les dates du projet, comme date_embauche, date_reservation, date_debut, date_fin_prevue, date_retour_reelle, date_paiement, date_frais, date_inscription et date_desinscription.

-Nous avons choisi DATE car le système de location fonctionne principalement par journée. Il n'est donc pas nécessaire de stocker l'heure exacte dans notre modèle.

Implémentation de l'Application

```
Démarrage de l'application de location de voitures...

  _ _ _ _ _
 /  | | | \  _ _
(   _ _ _ _ \
 = '-( )--( )-'
Bienvenue dans l'application de location de voitures !

=== Menu Principal ===
0. Quitter
=====
1. Inscription
2. Informations personnelles
3. Modifier les informations personnelles
4. Désinscription
5. Louer une voiture
6. Rapport annuel des ventes
7. Retourner un véhicule
=====
Choisissez une option: █
```

Conformément à l'énoncé, une application a été développée afin de permettre aux utilisateurs (Clients et Employés) d'interagir avec la base de données.

1. Choix Technologiques

- Langage : L'application a été entièrement développée en Python. Ce choix a été motivé par la lisibilité du langage et son excellente gestion des scripts en ligne de commande.
- Interface : L'interface utilisateur est déployée sous forme de console (CLI), favorisant la rapidité d'exécution et la simplicité.
- Communication Base de Données : La librairie mysql-connector-python a été utilisée pour établir la connexion avec le conteneur Docker MySQL et exécuter les requêtes SQL.

2. Architecture Logicielle

Afin de garantir un code lisible, évolutif et maintenable, l'application a été structurée selon le patron d'architecture MVC (Modèle-Vue-Contrôleur) couplé au patron DAO (Data Access Object) :

A. Le Modèle (src/modèle/)

Le modèle est composé d'une série de classes DAO (ex: ClientDAO, LocationDAO, VehiculeDAO).

- Rôle : Encapsuler toutes les requêtes SQL (CRUD). C'est la seule couche de l'application qui communique directement avec la base de données.
- Avantage : Si le schéma de la base de données évolue, seuls ces fichiers doivent être modifiés, sans impacter le reste de la logique applicative.

B. La Vue (src/vue/)

Le dossier contient le fichier interface.py (classe VueTerminal).

- Rôle : Gérer exclusivement les entrées et sorties (I/O) dans le terminal (les print() et input()). La vue ne fait aucun calcul métier ni requête SQL, elle se contente d'afficher les menus et de récupérer les saisies de l'utilisateur.

C. Le Contrôleur (src/contrôleur/)

Le contrôleur est représenté par la classe Application (application.py).

- Rôle : Il sert de chef d'orchestre. Il fait appel à la Vue pour récupérer les choix et informations de l'utilisateur, puis il invoque les DAO correspondants pour mettre à jour ou lire la base de données. C'est ici que réside la logique de vérification et d'enchaînement des étapes (ex: chercher le client, puis afficher les véhicules, puis créer la location, puis enregistrer le paiement).

3. Fonctionnalités Couvertes

L'application implémente avec succès l'ensemble du cycle de vie des utilisateurs et des locations :

1. Espace Client :
 - Inscription et Modification : Récolte des données personnelles via formulaire et insertion en base de données.
 - Désinscription (RGPD) : Le client peut se désinscrire. Ses données personnelles sont effacées (NULL/Anonyme) par l'application, et le booléen est_anonymise est activé, ce qui garantit le respect de la vie privée tout en conservant l'historique de ses paiements.
2. Processus de Location :
 - Affichage dynamique des véhicules dont le statut est "disponible" (via une Vue SQL).
 - Création conjointe de la location et enregistrement obligatoire de l'acompte (Paiement).
 - Clôture de la location (retour) : Mise à jour du kilométrage de retour et libération automatique du véhicule pour de futurs clients.
3. Espace Comptabilité (Sécurisé) :
 - Vérification de l'identité de l'employé (il doit obligatoirement exister dans la table d'héritage Comptable).
 - Preuve de permissions : Lors de la génération des rapports financiers annuels et mensuels, l'application se connecte à la base de données avec l'utilisateur SQL restreint comptable_user. Cela démontre l'efficacité des restrictions d'accès mises en place sur le serveur MySQL.

Implémentation SQL :

La partie SQL du projet est organisée dans le dossier init. Elle contient les fichiers nécessaires pour créer la base de données, insérer les données de test, créer les vues, ajouter les triggers et définir les permissions des utilisateurs.

1. Création des tables :

Le fichier 01_create_tables.sql contient la création de toutes les tables de la base de données.

Les principales tables créées sont :

- Agence
- CategorieVehicule
- Vehicule
- Client
- Reservation
- Location
- Paiement
- FraisSupplementaire
- Employe
- Comptable
- Agent_d_agence

Chaque table possède une clé primaire. Les relations entre les tables sont représentées avec des clés étrangères. Par exemple, un véhicule est lié à une catégorie et à une agence. Une réservation est liée à un client et à un véhicule. Une location est liée à un client, un véhicule, une agence de départ et une agence de retour.

2. Données de test :

Le fichier 02_insert_test_data.sql contient les données de test utilisées pour vérifier le fonctionnement de la base.

Ces données permettent de tester plusieurs situations :

- des agences différentes ;
- plusieurs catégories de véhicules ;
- des clients inscrits ;
- des véhicules disponibles ou loués ;
- des réservations confirmées ou annulées ;
- des locations terminées ou en cours ;
- des paiements effectués ou en attente ;
- des frais supplémentaires.

Ces insertions permettent de vérifier que les relations entre les tables fonctionnent correctement et que les données respectent le modèle final.

3. Vues :

Le fichier 03_views.sql contient trois vues principales.

3.1 Vue des véhicules disponibles

La vue VueVehiculesDisponibles permet d'afficher les véhicules dont l'état est disponible. Elle affiche aussi leur catégorie et l'agence dans laquelle ils se trouvent.

Cette vue est utile pour l'agent d'agence, car elle permet de voir rapidement quels véhicules peuvent être proposés à un client.

3.2 Vue du chiffre d'affaires mensuel

La vue `VueChiffreAffairesMensuel` calcule le chiffre d'affaires par mois à partir des paiements dont le statut est effectue.

Elle permet au comptable de consulter rapidement les montants encaissés chaque mois.

3.3 Vue du rapport annuel

La vue `VueRapportAnnuel` calcule le chiffre d'affaires total par année, toujours à partir des paiements effectués.

Elle sert à obtenir une vue globale des revenus de l'agence sur une année.

4. Triggers

Le fichier `04_triggers.sql` contient les triggers utilisés pour renforcer la cohérence de la base de données.

4.1 Vérification des réservations

Des triggers vérifient qu'un même véhicule ne peut pas être réservé deux fois sur des périodes qui se chevauchent.

Ils vérifient aussi que les dates d'une réservation sont cohérentes : la date de fin prévue ne peut pas être avant la date de début, et la date de réservation ne peut pas être après la date de début.

4.2 Vérification des locations

Des triggers empêchent qu'un même véhicule soit loué deux fois sur une même période.

Un autre trigger vérifie que lorsqu'une location est liée à une réservation, elle correspond bien au même client et au même véhicule. Cela évite de lier une location à une réservation qui ne lui correspond pas.

4.3 Vérification du kilométrage

Des triggers vérifient que le kilométrage de départ n'est pas négatif et que le kilométrage de retour n'est pas inférieur au kilométrage de départ.

Lors du retour d'un véhicule, le kilométrage actuel du véhicule est mis à jour automatiquement.

4.4 Mise à jour de l'état du véhicule

Lorsqu'une location est créée, l'état du véhicule devient loué.

Lorsqu'un véhicule est rendu, son état redevient disponible. Son agence actuelle est aussi mise à jour avec l'agence de retour.

4.5 Vérification des montants

Des triggers vérifient que les montants des paiements et des frais supplémentaires sont strictement positifs.

Cela évite d'enregistrer des paiements ou des frais avec des valeurs incorrectes.

4.6 Anonymisation des clients

Un trigger permet d'anonymiser un client lorsqu'il est désinscrit.

Ses informations personnelles sont remplacées par des valeurs anonymisées. Cela permet de conserver l'historique des locations et des paiements sans garder les données personnelles du client.

4.7 Gestion des rôles employés

Des triggers empêchent qu'un même employé soit à la fois Comptable et Agent_d_agence.

Cela permet de respecter la séparation entre les deux rôles définis dans le schéma.

5. Permissions

Le fichier 05_permissions.sql crée trois utilisateurs :

- agent_agence_user
- comptable_user
- admin_user

5.1 Agent d'agence

L'agent d'agence peut gérer les clients, les réservations et les locations. Il peut aussi consulter les véhicules, les agences et les catégories de véhicules.

5.2 Comptable

Le comptable peut gérer les paiements et les frais supplémentaires. Il peut aussi consulter les locations, les clients et les vues financières.

Ce rôle permet de séparer la partie financière du reste de l'application.

5.3 Administrateur

L'administrateur possède tous les droits sur la base de données.

Il peut donc gérer toutes les tables, les vues, les triggers et les utilisateurs.

Conclusion

Ce projet nous a permis de mettre en pratique l'ensemble des concepts théoriques abordés lors du cours de bases de données, en concevant un système fonctionnel de bout en bout.

Grâce à une modélisation rigoureuse et à l'utilisation avancée des fonctionnalités de MySQL (vues, déclencheurs et gestion fine des permissions), nous avons pu déléguer une grande partie de la logique métier et des contraintes d'intégrité directement au cœur de la base de données. L'application Python développée vient ensuite s'appuyer sur cette architecture solide via le patron de conception MVC et DAO, offrant une interface interactive et sécurisée aux différents utilisateurs (clients, agents et comptables).

Au final, le système implémenté répond à toutes les exigences initiales : il est robuste face aux erreurs, respecte la confidentialité des utilisateurs (anonymisation RGPD) et démontre une séparation claire des responsabilités entre le code applicatif et la base de données.